Software Requirements Specification: Credit Risk Scoring System

Version 1.0 · Sections 1 to 9 complete. Sections 6 to 9 have not yet been reviewed one at a time.

Document status and decisions in force

It upgrades a baseline credit-risk system (LightGBM model, Flask API, Streamlit dashboard) with a business-cost threshold optimizer, a fairness audit, and drift monitoring. The following decisions are in force. Appendix C lists the assumptions most worth confirming.

Area	Decision
Dataset	Freddie Mac Single-Family Loan-Level Dataset, sample files so the system runs locally
Target	Ever 90+ days delinquent within 24 months of first payment
Splits	Train on 2015–17 vintages, validate on 2018, out-of-time test on 2019, stress test on 2020–21 (see Section 2.6)
Threshold optimizer	TN and TP values fixed. Two sliders set FP and FN costs as a percentage of original loan balance. Profit reported in currency
Fairness	Audit only, using Fairlearn on non-geographic proxies (first-time buyer, number of borrowers, occupancy, loan purpose). Metrics update as the threshold moves. Mitigation and geographic demographic joins are stretch goals
Drift	Per-feature and score PSI. Two-tier alert: 0.1 warn, 0.2 retrain. Informational only. Real vintage drift plus a simulated shock script
Stack	Flask API, Streamlit client, Docker Compose, scripted pipeline, no MLflow

1. Scope and System Architecture

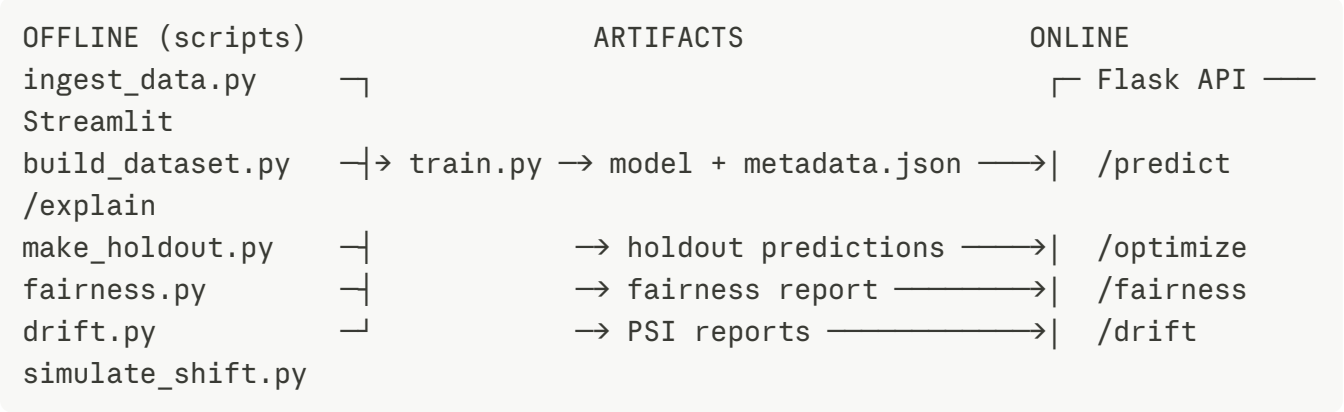
1.1 Purpose and scope

The system scores the default risk of 30-year fixed-rate mortgages, lets a risk manager tune the approval threshold against business costs, audits the model for group disparities, and monitors the scoring population for drift. It keeps the baseline's per-loan lookup with SHAP explanations. It is a portfolio project and is not intended for real lending decisions.

Out of scope for v1: group-specific threshold mitigation, geographic demographic joins, authentication, automated retraining, and a model registry.

1.2 Architecture

Three layers share one artifacts folder:



Offline pipeline (batch, run manually or via `make`). Builds the labelled dataset from origination and performance files, trains the LightGBM model, scores the validation and out-of-time sets, and writes all reports.

Artifacts (the contract between layers). The API never trains or computes heavy statistics. It only reads:

- the model file plus a `metadata.json` (model version, feature list, training vintages, default definition, default threshold)
- a holdout predictions table (loan ID, vintage, true label, probability, original balance, model features, fairness group columns)
- the fairness baseline and PSI report files

Online layer. A stateless Flask API serves predictions, explanations, and the optimizer, fairness, and drift views. The threshold sweep is a vectorized NumPy operation on the holdout table, which takes milliseconds, so it needs no locking or queue. Concurrent requests are safe because nothing is mutated. Streamlit stays a thin client with no direct data or model access, as in the baseline.

1.3 Non-functional requirements

- **Reproducibility:** one command rebuilds all artifacts from the raw files, and `metadata.json` records exactly what produced them.
- **Latency:** `/optimize` and `/fairness` respond in under 1 second on the validation set.
- **Leakage control:** features come only from origination-time fields. Performance-file data is used only to build the label.
- **Data licensing:** raw Freddie Mac data is never committed to the repository.

1.4 Known limitations to document

- Only loans Freddie Mac accepted are in the data, so rejected applicants are invisible to the model (selection bias).
- Fairness relies on proxies, not protected attributes.
- The 2020–21 stress set reflects an unusual interest-rate environment.

2. Data Pipeline

Draft for review. Field names, enumerations, and file formats were checked against the Freddie Mac Single-Family Loan-Level Dataset General User Guide, Release 48 (October 2026).

2.1 Inputs and ingestion

DP-01 Source files. The system uses the sample dataset: for each vintage year, one origination file (`sample_orig_YYYY.txt`) and one monthly performance file (`sample_perf_YYYY.txt`), packaged as `sample_YYYY.zip` . Each full vintage year is a simple random sample of 50,000 loans. Files are pipe-delimited and have no header row.

DP-02 Manual download. Access requires a free Clarity registration and sign-in, so downloading cannot be scripted. The user places the zip files in `data/raw/` . A script, `ingest_data.py` , unzips them, attaches column headers from a versioned schema file in the repository, and validates the result.

DP-03 Validation. Ingestion stops with a clear error if a required vintage file is missing, if a file's column count does not match the schema, if a loan identifier appears twice, or if the performance data for a vintage ends less than 24 months after the latest first-payment date in that vintage (the label window would be incomplete). The dataset release number is recorded in `metadata.json` .

2.2 Parsing

DP-04 Columns are loaded with explicit types, and only the columns needed for features and the label are kept in memory.

DP-05 Documented 'Not Available' codes are converted to null: FICO 9999; mortgage insurance %, CLTV, DTI, and LTV 999; number of units, number of borrowers, and property type 99; first-time homebuyer, occupancy, channel, and loan purpose 9; property valuation method 7. No imputation is applied, because LightGBM handles nulls natively. This replaces the baseline's median imputation.

2.3 Cohort selection

DP-06 Each loan's vintage is its origination year. The following loans are excluded, and the counts are recorded per vintage in `metadata.json`:

- Relief Refinance (HARP) loans, because their DTI is unavailable by design and they are a distinct high-LTV population. This is a v1 simplification.
- Loans whose final Zero Balance Code is 96 (confirmed underwriting or servicing defect), because their outcome is ambiguous.
- Loans with no performance records.

The Standard Dataset contains only fixed-rate, fully amortizing loans, so no adjustable-rate handling is needed.

2.4 Target label

DP-07 `default_24m` is 1 when, within 24 months of the first payment date, a loan has either of the following, and 0 otherwise:

- a Current Loan Delinquency Status of 03 or higher (90+ days delinquent) or RA (REO acquisition); or
- a Zero Balance Code of 02, 03, 09, 15, or 16 (credit-event terminations).

DP-08 Label rules and edge cases:

- The window is calculated as the performance Period minus the First Payment Date. The Loan Age field is not used, because it restarts for modified loans.
- Rows with delinquency status XX (not available) are ignored.
- Loans that pay off or mature voluntarily (code 01) inside the window are labelled 0.
- Pandemic forbearance affects the 2020–21 stress vintages. Delinquencies are counted as reported, and this is documented as a limitation.

2.5 Features

DP-09 Features are limited to origination-time fields. Performance-file fields are used only to build the label.

Group	Included
Numeric	Classic FICO, mortgage insurance %, number of units, original CLTV, original DTI, original UPB, original LTV, original interest rate, original loan term, number of borrowers
Categorical	First-time homebuyer, occupancy status, channel, prepayment penalty indicator, property state, property type, loan purpose, seller name, special eligibility program, property valuation method, interest-only indicator, super conforming flag
Derived	`cltv_minus_ltv` (secondary financing) and `rate_vs_quarter_median` (original rate relative to the median rate of the loan's origination quarter, a simple measure of the rate environment)

Excluded	Reason
Loan identifier, Pre-HARP loan sequence number	Identifiers only
First payment date, maturity date	Would act as a vintage proxy; original loan term is used instead
3-digit postal code, MSA	High cardinality and a geographic proxy for protected groups; property state is kept
HARP indicator	Used as a cohort filter (DP-06)
VantageScore 4.0	v1 uses a single credit score, Classic FICO
Servicer name and every other performance-file field	Post-origination information that would leak the outcome

The fairness proxies (first-time homebuyer, number of borrowers, occupancy, loan purpose) are ordinary underwriting inputs and remain model features. The fairness section will define how the audit treats them.

2.6 Time-based splits

DP-10 Splits follow origination vintage:

Split	Vintages	Use
Train	2015 to 2017	Model fitting; tuning by stratified 5-fold cross-validation inside this set only
Validation	2018	Early stopping, default threshold, optimizer sweep
Out-of-time test	2019	Reported once as the honest performance estimate
Stress	2020 to 2021	Reported once; real drift and rate-environment shift

There is no random split across vintages. Each full vintage year contributes up to 50,000 loans before exclusions.

2.7 Outputs

DP-11 The pipeline writes:

- `data/processed/{train,valid,oot,stress}.parquet`: the cleaned feature tables with labels.
- `artifacts/holdout_predictions.parquet`: for the validation, test, and stress splits, the loan ID, split, vintage, label, predicted probability, original UPB, and the fairness attribute columns.
- `artifacts/loss_severity.json`: the mean of Actual Loss divided by original UPB over defaulted training-vintage loans that have a loss value. It sets the default for the false-negative cost slider. Actual Loss exists only for Zero Balance Codes 02, 03, 09, and 15, and is null for loans disposed shortly before the performance cutoff, so it reflects resolved defaults only.
- Additions to `metadata.json`: dataset release number, vintages per split, exclusion counts, default rate per split, label definition version, and the feature list.

DP-12 The optimizer sweeps the validation split by default and also reports the resulting metrics on the out-of-time split, so the chosen threshold is checked on data it was not tuned on.

2.8 Amendment to Section 1

The `download_data.py` step in the architecture diagram is replaced by the manual download and `ingest_data.py` described in DP-02.

3. Model Training and Scoring

Draft for review. This section covers the model, its evaluation, explanations, and the scoring behaviour the API relies on. Endpoint contracts are specified in the API section.

3.1 Model and training

MD-01 Model. A LightGBM binary classifier. Categorical features from DP-09 use LightGBM's native categorical handling. The category levels are fixed from the training split and saved in `metadata.json`; a level not seen in training is treated as null at scoring time.

MD-02 No class weighting or resampling. The baseline used balanced class weights and tested SMOTE and undersampling. Class weights inflate predicted probabilities, and the baseline found that SMOTE did not improve validation scores. This system trains without weights or resampling, so the predicted probability stays interpretable as an estimated default probability. The imbalance is handled by choosing the decision threshold from the business cost function (Section 3.3), not by reweighting.

MD-03 Tuning. A randomized search of 20 candidates, scored by ROC AUC with stratified 5-fold cross-validation inside the training split only. The search covers `num_leaves`, `max_depth`, `learning_rate`, `min_child_samples`, `subsample`, `colsample_bytree`, and `reg_lambda`. The best parameters are then refit on the full training split with early stopping (50 rounds, log loss) on the validation split.

MD-04 Reproducibility. A fixed random seed is used throughout. `metadata.json` records the seed, the chosen parameters, library versions, the training timestamp, and the category levels.

3.2 Evaluation

MD-05 Metrics. For the validation, out-of-time, and stress splits, the pipeline reports ROC AUC, Gini (2 × AUC − 1), KS statistic, PR AUC, Brier score, and log loss. It also reports a score-decile table (loans, defaults, observed default rate, mean predicted probability) and a calibration table. Results are written to `artifacts/model_report.json` and a readable `reports/model_report.md`.

MD-06 Sanity gates. The training run fails if out-of-time AUC falls below a configurable floor (initially 0.60), which would indicate a bug rather than a weak model. It warns if out-of-time AUC is more than 0.05 below validation AUC. Both values live in `config.yaml`.

3.3 Default decision threshold

MD-07 Threshold. The default threshold is the probability cut-off that minimizes total cost on the validation split under the default cost parameters. The false-negative cost is the mean loss severity (from `loss_severity.json`) times original UPB. The false-positive cost is a percentage of original UPB, whose default is set in the Optimizer section. A loan with a probability at or above the threshold is classified as predicted default (reject). The threshold is stored in `metadata.json`.

MD-08 Shared cost code. The cost function lives in one module (`cost.py`) that both the offline pipeline and the API import, so the offline default and the interactive optimizer can never disagree.

3.4 Explainability

MD-09 Local explanations. A single loan's explanation uses LightGBM's built-in contribution output (`pred_contrib=True`), which returns exact TreeSHAP values per feature plus a base value. This avoids a separate `shap` dependency in the API and handles categorical features directly. The response lists the top 10 features by absolute contribution, each with the loan's value and its contribution.

MD-10 Global importance. Mean absolute contribution over a random sample of 5,000 validation loans, computed offline and saved to `artifacts/global_importance.json`.

MD-11 Units. Contributions are in log-odds, not probability. The dashboard must label them accordingly.

3.5 Scoring behaviour

MD-12 Live scoring of stored loans. The API loads the model and the holdout feature table at startup. For a requested loan ID it scores the stored features live and returns the probability, the decision at the current default threshold, and the model version. Scoring new, user-entered loans is out of scope for v1.

MD-13 Startup self-check. The API re-scores 100 sampled loans and compares them with the probabilities stored at training time. If any difference exceeds 1e-6, the API refuses to start, because the model and data artifacts do not match.

MD-14 Served loans. Only loans from the validation, out-of-time, and stress splits can be requested. Training loans are not served, so users never see in-sample scores. An unknown loan ID returns a not-found error.

MD-15 Concurrency. The model is loaded once per worker process and never modified. The API runs under gunicorn with synchronous worker processes and no shared mutable state, so concurrent requests are safe.

3.6 Artifacts produced

Artifact	Contents
`artifacts/model.txt`	LightGBM model in its native text format, which avoids pickle version and security problems
`artifacts/metadata.json`	Adds seed, parameters, library versions, category levels, default threshold, cost parameters used, and training timestamp
`artifacts/model_report.json`	Metrics, decile table, and calibration table per split
`artifacts/global_importance.json`	Global feature importance (MD-10)

3.7 Amendment to DP-11

The holdout table must also contain the model's feature columns, because the explanation endpoint needs them. With up to 50,000 loans per vintage year this is roughly 200,000 rows across the validation, test, and stress splits, which fits comfortably in memory.

4. Threshold Optimizer

Draft for review. This section defines the business cost model, the two cost sliders, and how the optimal threshold is computed. Endpoint contracts are specified in the API section.

4.1 Cost model

OP-01 Definitions. Each loan has an original balance B, a label y (1 = default), and a predicted probability p. A loan is predicted to default, and therefore rejected, when p is at or above the threshold t. Two business parameters are each expressed as a percentage of original balance:

- **g, the false-positive cost:** the margin lost by rejecting a loan that would have repaid (the opportunity cost).
- **L, the false-negative cost:** the loss from approving a loan that defaults.

Rejecting a defaulting loan (true positive) costs nothing, and approving a good loan (true negative) earns g. Because these two values follow from the definition, they are not user parameters. This resolves the earlier 'TN and TP fixed' assumption, and it differs from the baseline's +1 and +0.2 scoring.

OP-02 Dollar results. Sums run over the loans in the selected split. B_TP means the total original balance of true-positive loans, and so on.

```
profit(t)          = g*B_good - g*B_FP(t) - L*B_FN(t)
approve_all_profit = g*B_good - L*B_default
net_benefit(t)     = profit(t) - approve_all_profit
                   = L*B_TP(t) - g*B_FP(t)
```

Maximizing profit, maximizing net benefit, and minimizing total cost (g × B_FP + L × B_FN) all pick the same threshold. Net benefit is the headline figure: the money saved compared with approving every loan.

OP-03 Caveats shown in the UI. The costs are flat percentages of balance, with no time value of money, prepayment, or servicing costs. Results describe the evaluated sample (up to 50,000 loans per vintage year), not Freddie Mac's whole book. Currency is USD.

4.2 Inputs

OP-04 Sliders.

Slider	Range	Step	Default
Cost of false positive (g)	0.5% to 10% of balance	0.5%	2.0%, an illustrative placeholder set in `config.yaml`, not estimated from the data
Cost of false negative (L)	5% to 60% of balance	1%	Mean loss severity from `loss_severity.json`, rounded and clamped to the range

OP-05 Mode. Optimal mode finds the threshold by sweep. Manual mode takes a threshold t between 0 and 1 supplied by the user, so the dashboard and the fairness view can show the effect of any threshold, not only the best one.

OP-06 Split. The sweep runs on the selected split, which defaults to validation. Metrics at the resulting threshold are always reported for all three splits (OP-11).

4.3 Computation

OP-07 Exact sweep. At API startup, each split is sorted by probability in descending order, and the cumulative balance of defaulting loans and of good loans is precomputed. For given g and L, rejecting the top k loans gives net benefit L × cumTP(k) − g × cumFP(k), and the optimum is the best k, including k = 0 (reject nothing). The sort does not depend on g or L, so each request is a single vectorized pass over at most about 100,000 values.

OP-08 Ties and edge cases. Only the end of each group of equal probabilities is a candidate k. If several candidates tie, the one that rejects the fewest loans wins. The reported threshold is the lowest probability among rejected loans. If rejecting nothing is optimal, the response sets `reject_none` to true and the threshold to 1.0.

OP-09 Analytic cross-check. Because the model is trained without class weights (MD-02), a well-calibrated model should have its optimal threshold near g / (g + L). With g = 2% and L = 25%, that is about 0.074. The response includes this value. If the swept threshold is more than twice or less than half the analytic one, the UI shows a calibration warning for that split. The factor of two is configurable.

4.4 Outputs

OP-10 Metrics. For the chosen threshold and split: confusion counts (TP, FP, TN, FN), precision (TP / (TP + FP)), recall as the default capture rate (TP / (TP + FN)), rejection rate, the observed default rate among approved loans, profit, approve-all profit, net benefit, and net benefit per loan.

OP-11 Three-split table. A table with one row each for validation, out-of-time, and stress, showing the same metrics at the same threshold. This shows how a threshold tuned on 2018 holds up on 2019 and on 2020–21. The optimum is computed for the selected split only; the other rows use that threshold unchanged.

OP-12 Curve. A net-benefit-versus-threshold curve for the chart, with at most 200 points evenly spaced in rank, plus the optimum marked.

4.5 Validation and edge cases

OP-13 Values outside the slider ranges, non-numeric values, and manual thresholds outside 0 to 1 are rejected with a clear error.

OP-14 If a split has no defaulting loans or no good loans, the API returns a 'not computable' message instead of dividing by zero. Precision is null when no loans are rejected.

OP-15 Latency and state. Each request is a pure function of the split, g, L, mode, and threshold over read-only arrays. It answers in under 1 second (Section 1.3) and needs no locking or caching.

4.6 Items resolved from earlier sections

- **Default false-positive cost:** 2% of original balance, illustrative and configurable.
- **MD-07 default threshold:** computed offline by the shared `cost.py` using the default g and the mean loss severity as L on the validation split. It equals the optimizer's Optimal mode at default slider values.

5. Fairness Audit

Draft for review. This section defines which groups are compared, which metrics are computed, and how the audit reacts to the optimizer's threshold. Endpoint contracts are specified in the API section.

5.1 Framing

FA-01 What the audit is. The audit compares model outcomes across groups of loans defined by four proxy attributes. These are not legally protected classes: the dataset contains no race, sex, or age. They are used because they may correlate with protected characteristics (for example, borrower count with marital status, and first-time buyer status with age and wealth). A flagged gap is a prompt to investigate, not proof of discrimination.

FA-02 Legitimate risk drivers. Occupancy and loan purpose are also genuine risk factors, so some disparity is expected. The dashboard shows this caution beside every flag.

5.2 Attributes and groups

Attribute	Source field	Groups
First-time homebuyer	First Time Homebuyer Indicator	Yes, No, Unknown
Borrower count	Number of Borrowers	1 borrower, 2 or more, Unknown
Occupancy	Occupancy Status	Primary residence, Investment property, Second home, Unknown
Loan purpose	Loan Purpose	Purchase, Refinance cash-out, Refinance no cash-out, Other or unknown

FA-03 Group columns. The pipeline creates one group column per attribute and stores them in the holdout table. The attribute list and group labels live in `config.yaml`. Borrower count is bucketed into 1 and 2 or more because Freddie Mac changed its enumeration in 2018Q2 (earlier, 2 meant more than one borrower; later, 2 means exactly two). The 'Other or unknown' loan-purpose group combines 'refinance, not specified' and 'not available'.

FA-04 Unknown groups. Unknown groups are shown in the tables but excluded from every gap metric and flag.

5.3 Metrics

FA-05 Per-group metrics. At the chosen threshold and split, each group reports: loans, defaults, observed default rate, mean predicted probability, rejection rate, approval rate, default capture rate (the share of actual defaulters rejected), and false rejection rate (the share of good loans rejected). Every rate carries a 95% Wilson confidence interval.

FA-06 Summary measures. For each attribute, over the rated groups:

Measure	Definition	Flag when
Disparate impact ratio	Lowest approval rate divided by highest approval rate	Below 0.80 (the four-fifths rule)
Demographic parity difference	Highest minus lowest rejection rate	Reported only
Equalized odds difference	The larger of the biggest gap in default capture rate and the biggest gap in false rejection rate (Fairlearn's definition)	Above 0.10
Calibration ratio	Observed default rate divided by mean predicted probability, per group	Outside 0.80 to 1.25

The flag levels are initial values and are configurable. The calibration ratio is only judged for groups with at least 30 defaults.

FA-07 Size rule. A group is rated only if it has at least 500 loans and at least 30 defaults (both configurable), because rates built on few defaults are noisy. Smaller groups are shown, marked 'too small', and left out of the gap metrics.

FA-08 Flag detail. Each flag names the groups behind it, for example 'Investment property versus Primary residence'.

5.4 Computation and the role of Fairlearn

FA-09 Live computation. The API computes group counts with vectorized NumPy operations on the holdout table, so the audit recalculates in milliseconds whenever the threshold changes. Fairlearn is not called at request time.

FA-10 Offline baseline. `fairness.py` uses Fairlearn's `MetricFrame` to compute the same metrics at the default threshold for all three splits and writes `artifacts/fairness_baseline.json`. For the disparate impact ratio the favourable outcome is approval, so Fairlearn is given approval as the selected outcome.

FA-11 Cross-check test. An automated test asserts that the API's calculations match Fairlearn's at the default threshold, within a tolerance of 1e-9. This keeps the fast implementation honest.

5.5 Interaction with the optimizer

FA-12 The audit uses the same split and the same threshold as the optimizer, in Optimal or Manual mode (OP-05, OP-06). Moving a cost slider or the manual threshold therefore updates the audit. The response states the threshold used and includes the baseline values at the default threshold for comparison.

5.6 Outputs

FA-13 For each attribute the response contains the group table, the summary measures, the flags with their reasons, and a short interpretation note from `config.yaml` (including the FA-02 caution). The dashboard's Fairness and Bias Audit tab displays these.

5.7 Edge cases

FA-14 Undefined rates. A group with no defaults has a null default capture rate. A group with no good loans has a null false rejection rate. Null values are left out of gap calculations.

FA-15 Too few groups. An attribute with fewer than two rated groups returns 'not enough rated groups' instead of metrics.

FA-16 Reject-nothing threshold. If the threshold rejects no loans, every rejection rate is 0, the disparate impact ratio is 1.0, and nothing is flagged.

FA-17 Latency and state. Each request is stateless and answers in under 1 second (Section 1.3).

5.8 Out of scope for v1

- Mitigation, such as group-specific thresholds. In real lending these can amount to disparate treatment, so v1 audits only. Fairlearn's `ThresholdOptimizer` as a what-if comparison is a stretch goal.
- Intersectional groups (for example, first-time buyers who are also investors).
- Geographic proxies and joins to Census data.

5.9 Amendments to earlier sections

- **DP-09:** the number of borrowers is capped at 2 as a model feature (2 means 2 or more), so its meaning is the same before and after the 2018Q2 enumeration change. Without this, validation loans from 2018 would mix two meanings.

- **DP-11:** the holdout table also contains the four group columns from FA-03.

6. Drift Monitoring

Draft for review. This section defines how the system detects that the loans being scored no longer resemble the loans the model was trained on, and how that is simulated and reported.

6.1 Definitions

DR-01 PSI. The Population Stability Index compares the share of loans in each bin now (a) with the share in a reference set (e):

```
PSI = sum over bins of (a - e) × ln(a / e)
```

DR-02 Bins. Numeric features use 10 bins whose edges are the deciles of the reference data, plus a separate 'missing' bin. Categorical features use one bin per category seen in the reference, plus 'unseen' and 'missing' bins. The model score uses 10 bins with edges from the reference score deciles. All edges and reference shares are saved so that every later comparison uses identical bins.

DR-03 Smoothing. Any bin share below 0.0001 is raised to 0.0001 before the calculation, so an empty bin never produces an infinite value.

6.2 References and windows

DR-04 References. The feature reference is the training split. The score reference is the validation split's scores, not the training scores, because the model has fitted the training loans and their scores do not represent unseen loans.

DR-05 Windows. PSI is computed for the validation, out-of-time, and stress splits, and for each origination quarter from 2018Q1 to 2021Q4. The quarterly series feeds the dashboard's trend chart. Each window yields a PSI for the score and for every model feature.

DR-06 Minimum sample. A window with fewer than 1,000 loans reports 'insufficient data' instead of a PSI.

6.3 Status rules

DR-07 Bands.

PSI	Status	Meaning
Below 0.10	Stable	No action
0.10 to 0.20	Warning	A shift worth watching
Above 0.20	Retrain recommended	The scored population differs materially from training

DR-08 Overall status. The status of a window is the worst band among the score PSI and the PSI of the 10 most important features (MD-10). The report lists which items triggered it. Other features are reported but do not set the status.

DR-09 Structural exclusions. Features whose missingness changes by design are listed in `config.yaml`. They are still reported, marked 'structural', but do not set the overall status. Initially this list holds property valuation method, which Freddie Mac populates only for loans originated from 2017 onward, so the 2015–16 training vintages differ from later ones for reasons unrelated to borrowers.

DR-10 Expected behaviour. The stress vintages (2020–21) are expected to show a PSI above 0.20 on original interest rate, because the rate environment changed sharply. If they do not, the binning should be checked before the monitor is trusted.

6.4 Simulated shocks

DR-11 Scenarios. `simulate_shift.py` applies a named shock to a copy of the out-of-time split, rebuilds the derived features, scores the copy with the trained model, and computes PSI against the same references. Every loan in the copy is shifted.

Scenario	Shock	Mild	Moderate	Severe
Debt stress	Original DTI increased, capped at 65	+3 points	+6 points	+10 points
Credit deterioration	Classic FICO decreased, kept within 300 to 850	-15 points	-30 points	-50 points
Rate shock	Original interest rate increased	+0.5 points	+1.0 points	+2.0 points
Leverage squeeze	Original LTV and CLTV increased, capped at 100	+3 points	+6 points	+10 points

The shock sizes are initial values and live in `config.yaml`.

DR-12 Shared feature code. Derived features are rebuilt by the same `features.py` module used in training, so the simulation cannot drift from production logic.

DR-13 Precomputed grid. The 4 scenarios at 3 severities give 12 results, all written offline. The dashboard selects one of them. There is no live simulation in v1.

DR-14 What simulation reports. Simulated loans have no true outcomes, so simulation reports PSI, the shift in the score distribution, and the change in mean predicted probability (a proxy for the expected default rate). It produces no AUC, profit, or fairness results.

6.5 Artifacts

Artifact	Written by	Contents
`artifacts/reference_bins.json`	`train.py`	Bin edges or categories and reference shares for every feature and the score
`artifacts/psi_report.json`	`drift.py`	PSI per window, status, triggers, and the quarterly series
`artifacts/psi_simulated.json`	`simulate_shift.py`	The 12 scenario results

6.6 Alerts

DR-15 The API returns the status and its triggers, and the dashboard shows them as a banner (Section 8). Alerts are informational only: nothing retrains automatically. For a 'Retrain recommended' status the dashboard shows the next step, which is to add newer data and rerun the full pipeline (`make all`), then compare the new reports.

6.7 Edge cases and exclusions

DR-16 A feature missing from the saved bins is a hard error when the report is built, because it signals a schema mismatch. A category with no reference share but a current share is handled by smoothing and contributes a large PSI by design. A constant feature has one bin, a PSI of 0, and is reported as 'constant'.

DR-17 Out of scope. Automated retraining; performance monitoring on new loans, because default labels take 24 months to mature; and concept-drift detection.

7. API Specification

Draft for review. The Flask API is read-only: it serves model scores, explanations, and the precomputed reports, and it computes only the optimizer and fairness views (Sections 4 and 5).

7.1 Conventions

AP-01 Format. All endpoints live under `/v1`, use GET, return JSON in UTF-8, and change no state.

AP-02 Provenance. Every successful response includes `model_version` and `data_release`, both taken from `metadata.json`.

AP-03 Names. Splits are named `valid`, `oot`, and `stress`. Loan IDs are Freddie Mac loan identifiers. Cost parameters are passed in percent units, so `2.0` means 2% of original balance.

AP-04 Errors. Errors use one shape, with a machine-readable code:

```
{ "error": { "code": "invalid_parameter", "message": "fp_cost_pct must be between 0.5 and 10", "details": { "parameter": "fp_cost_pct" } } }
```

HTTP status	Code	When
400	`invalid_parameter`	A value is out of range, not numeric, missing, or the parameter is unknown (typos are rejected rather than ignored)
404	`loan_not_found`	The loan ID is not in the validation, out-of-time, or stress splits (MD-14)
422	`not_computable`	A split has no defaulting loans or no good loans (OP-14), or a group-based result cannot be formed
503	`artifacts_unavailable`	A required artifact is missing or failed the startup self-check
500	`internal_error`	Anything unexpected. The response carries a request ID that also appears in the logs

AP-05 Access. There is no authentication in v1. The API is meant to run locally or on a private network, and only the dashboard calls it (Section 9).

AP-06 Caching headers. Endpoints that serve fixed artifacts (`/meta` , `/model-report` , `/explain/global` , `/drift`) send `Cache-Control: max-age=60` . Computed endpoints send `no-store` .

AP-07 Performance. On a laptop, `/optimize` and `/fairness` answer in under 1 second, and `/predict` and `/explain` in under 200 milliseconds.

AP-08 Versioning. A breaking change to any contract creates `/v2` . `/v1` is not altered in place.

7.2 Endpoints

Endpoint	Purpose	Parameters	Main response fields
`GET /v1/health`	Readiness check	None	`status` , `model_version` , `data_release` , loans loaded per split
`GET /v1/meta`	Settings the dashboard needs at start-up	None	`default_threshold` , default `fp_cost_pct` and `fn_cost_pct` , slider ranges, splits, fairness attributes and groups, PSI bands, feature list
`GET /v1/loans`	List loans for a picker	`split` (required), `limit` (1 to 500, default 100), `offset` , `search` (loan ID prefix)	`total` , `items` (each with `loan_id` , `vintage` , `actual_outcome`)
`GET /v1/loans/{loan_id}`	One loan's details	None	`split` , `vintage` , `original_upb` , `actual_outcome` , `features` (name to value)
`GET /v1/predict/{loan_id}`	Score a stored loan (MD-12)	`threshold` (optional, 0 to 1)	`probability` , `threshold` , `threshold_source` (default or supplied), `decision` (approve or reject)

Endpoint	Purpose	Parameters	Main response fields
`GET /v1/explain/{loan_id}`	Local explanation (MD-09)	None	`base_value`, `probability`, `contributions` (top 10, each with `feature`, `value`, `contribution_log_odds`), `other_contribution_log_odds` for the remaining features
`GET /v1/explain/global`	Global importance (MD-10)	`top` (default 20)	`features` with `mean_abs_contribution`
`GET /v1/optimize`	Threshold optimizer (Section 4)	`split` (default `valid`), `fp_cost_pct`, `fn_cost_pct`, `mode` (`optimal` or `manual`), `threshold` (required when manual)	See 7.3
`GET /v1/fairness`	Fairness audit (Section 5)	The same parameters as `/optimize`, plus `attribute` (optional; all attributes if omitted)	`threshold_used`, `baseline`, `attributes` (each with groups, summary measures, flags, interpretation)
`GET /v1/drift`	Drift report (Section 6)	`source` (`real` or `simulated`). Real: `window` (`valid`, `oot`, `stress`, or `quarterly`). Simulated: `scenario` and `severity`	`status`, `triggers`, `score_psi`, `features` (PSI, band, structural flag), score histogram for reference and current, quarterly series where requested

Endpoint	Purpose	Parameters	Main response fields
`GET /v1/model-report`	Model metrics (MD-05)	None	The contents of `model_report.json`

AP-09 Shared threshold logic. `/fairness` resolves its threshold with the same function as `/optimize`, so the two endpoints can never disagree for the same parameters (FA-12). The optimizer's `fp_cost_pct` and `fn_cost_pct` defaults come from `/meta`.

AP-10 Cross-endpoint rule. `/predict` accepts an optional `threshold` so the dashboard can show a loan's decision at the threshold the user chose in the optimizer, not only at the default.

7.3 Example: optimizer response

The values below are illustrative only.

```
{
  "model_version": "2026-11-07-a1b2c3",
  "data_release": "48",
  "split": "valid",
  "inputs": { "fp_cost_pct": 2.0, "fn_cost_pct": 25.0, "mode": "optimal" },
  "threshold": 0.081,
  "reject_none": false,
  "analytic_threshold": 0.074,
  "calibration_warning": false,
  "metrics": {
    "tp": 0, "fp": 0, "tn": 0, "fn": 0,
    "precision": 0.0, "recall": 0.0, "rejection_rate": 0.0,
    "approved_default_rate": 0.0,
    "profit": 0.0, "approve_all_profit": 0.0,
    "net_benefit": 0.0, "net_benefit_per_loan": 0.0
  },
  "splits": [
    { "split": "valid", "metrics": { } },
    { "split": "oot", "metrics": { } },
    { "split": "stress", "metrics": { } }
  ],
  "curve": [ { "threshold": 0.01, "net_benefit": 0.0 } ]
}
```

The `splits` array carries the same metric keys as `metrics` (OP-10, OP-11). The `curve` array holds at most 200 points (OP-12).

8. Dashboard (UI/UX)

Draft for review. The Streamlit dashboard is a thin client: it calls the API and never reads the model or the data directly.

8.1 Structure

UI-01 Layout. One page with a sidebar and five tabs.

Tab	Purpose	API calls
Loan Lookup	Pick a loan and see its score, decision, and the reasons behind it	`/loans`, `/loans/{id}`, `/predict/{id}`, `/explain/{id}`
Business Optimizer	Tune the two cost sliders and see the best threshold and its value	`/optimize`
Fairness and Bias Audit	See whether outcomes differ across groups at the chosen threshold	`/fairness`
Drift Monitor	See whether the scored population has shifted, including simulated shocks	`/drift`
Model and Data	Model quality, global importance, data lineage, and limitations	`/model-report`, `/explain/global`, `/meta`

UI-02 Shared controls. The sidebar holds the controls that drive both the Business Optimizer and the Fairness tabs: split, cost of false positive, cost of false negative, mode (Optimal or Manual), and the manual threshold. Changing a control in one tab changes it in the other, which is what makes FA-12 visible to the user. The sidebar also shows the model version and data release from `/health`.

8.2 Tab behaviour

UI-03 Loan Lookup. The user chooses a split, then a loan from a searchable list (`/loans` with `search`). The tab shows the estimated default probability, the decision at the sidebar threshold (`/predict` with `threshold`), the actual outcome, and a bar chart of the top 10 contributions. The chart is labelled 'log-odds contributions' (MD-11), with bars that raise risk in one colour and bars that lower it in another, plus the combined 'other features' bar so the bars reconcile to the score.

UI-04 Business Optimizer. Shows the two sliders' effect as headline cards (threshold, net benefit, profit versus approving everyone, default capture rate, rejection rate), the net-benefit-versus-threshold curve with the optimum marked, and the three-split table (OP-11). The calibration warning (OP-09) and the caveats (OP-03) appear as visible notes, not tooltips.

UI-05 Fairness and Bias Audit. One attribute at a time, chosen from a selector, with the group table, a rejection-rate chart, and a chart of default capture rate and false rejection rate with confidence-interval whiskers. Flags appear in a panel that names the groups behind each flag, next to the caution from FA-02. A 'baseline versus current' view shows how the metrics moved from the default threshold. Too-small and Unknown groups are shown greyed and labelled.

UI-06 Drift Monitor. A status banner at the top, a bar chart of PSI by feature with the 0.10 and 0.20 lines drawn, an overlay of reference and current score distributions, and a quarterly PSI trend. A scenario selector (scenario and severity) switches the view to a simulated shock and labels it 'simulated' everywhere it appears (DR-13, DR-14).

UI-07 Model and Data. Shows model metrics by split, decile and calibration tables, the global importance chart, the data release and label definition, and the list of known limitations from Section 1.4.

8.3 Alerts

UI-08 Banner. The drift banner is green for Stable, amber for Warning, and red for Retrain recommended. It names the triggering items and shows the next step from DR-15. The banner text always states the status in words, so it is not conveyed by colour alone.

8.4 Behaviour and quality

UI-09 Responsiveness. Sliders send a request when released, not on every movement. Identical requests are cached for 60 seconds. Each interaction should complete in under 2 seconds on a laptop, and a spinner shows while a request is in flight.

UI-10 Errors. If the API is unreachable, a banner explains that and offers a retry. API error messages (AP-04) are shown in plain language, and the previous successful result stays on screen, marked as stale.

UI-11 Units and formats. Money is shown in USD with thousands separators. Cost sliders show percent of original balance. Probabilities are shown as percentages with one decimal. Explanations are always labelled log-odds.

UI-12 Accessibility. Charts have titles and axis labels, status never relies on colour alone, and every group in a chart is also in a table.

UI-13 No data copies. The dashboard stores nothing on disk and holds no loan-level data beyond the response it is currently displaying.

9. Deployment, Testing, and Project Structure

Draft for review. This section turns the earlier sections into a repository layout, a run model, and a test plan that `implementation.md` will follow.

9.1 Repository layout

DT-01 The repository is organized as follows. Shared code lives in one `common` package that is installed into the pipeline, API, and dashboard environments, so the cost function (MD-08) and the feature logic (DR-12) exist exactly once.

```
credit-risk-scoring/
  config.yaml
  Makefile
  docker-compose.yml
  README.md
  requirements/        pipeline.txt  api.txt  dashboard.txt
  schema/              origination_columns.json  performance_columns.json
  data/
    raw/               user-supplied zip files (not committed)
    processed/         parquet tables (not committed)
  artifacts/           model, reports, holdout table (not committed)
  reports/             model_report.md
  src/
    common/            config.py  features.py  cost.py  psi.py  groups.py
    pipeline/          ingest_data.py  build_dataset.py  train.py
                       make_holdout.py  fairness.py  drift.py
simulate_shift.py
  api/                 app.py  loaders.py  routes/  Dockerfile
  dashboard/           app.py  client.py  tabs/  Dockerfile
  tests/               unit/  integration/
```

9.2 Configuration

DT-02 All tunable values live in `config.yaml`, with no numbers hard-coded elsewhere: file paths, random seed, vintages per split, tuning iterations, sanity gates (MD-06), cost slider ranges and default false-positive cost (OP-04), calibration-warning factor (OP-09), fairness attributes, group labels, flag levels and size rule (FA-03 to FA-07), PSI bands and structural exclusions (DR-07, DR-09), shock sizes (DR-11), and API limits (AP-04).

9.3 Running the system

DT-03 Make targets.

Command	What it does
`make ingest`	Unzips and validates the raw files (DP-02, DP-03)
`make dataset`	Builds labels, features, and splits (Section 2)
`make train`	Tunes, trains, evaluates, and writes the model, thresholds, bins, and holdout table (Section 3)
`make audit`	Writes the Fairlearn baseline (FA-10)
`make drift`	Writes the PSI report (DR-05)
`make simulate`	Writes the 12 simulated scenarios (DR-13)
`make all`	Runs everything above in order
`make up`	Starts the API and dashboard with Docker Compose
`make test`	Runs the unit and integration tests

DT-04 Docker Compose. Two services, `api` (port 8000) and `dashboard` (port 8501). Both mount `artifacts/` read-only. The dashboard reaches the API through an `API_URL` environment variable. The API runs under gunicorn with two synchronous workers (MD-15). The API has a health check, and the dashboard starts only after it passes. Ports are published to localhost only.

DT-05 Environments. Python version and library versions are pinned in the three requirements files. The API environment contains no training libraries, and the dashboard environment contains neither LightGBM nor Fairlearn.

DT-06 Logging. The API writes one structured log line per request (request ID, endpoint, duration, status). At start-up it logs the artifact versions and the result of the self-check (MD-13).

9.4 Test plan

DT-07 Unit and integration tests.

ID	Area	What must hold
TS-01	Optimizer	The sweep returns the same threshold and net benefit as a brute-force search over every candidate threshold on random data, including ties and reject-nothing
TS-02	Cost model	Net benefit equals profit minus approve-all profit for random inputs (OP-02)
TS-03	Label	On synthetic performance records: a loan reaching 90 days late within the window is 1, one reaching it after the window is 0, a modified loan whose Loan Age restarted is still windowed by dates, code 96 loans are excluded, and code 01 loans are 0 (DP-07, DP-08)
TS-04	Leakage	The feature list contains no performance-file field (DP-09)
TS-05	PSI	Identical data gives 0, a hand-worked two-bin example matches, and an empty reference bin does not produce an infinite value (DR-01 to DR-03)
TS-06	Simulation	Within each scenario, PSI on the shocked feature never decreases as severity rises, and at least one severe scenario exceeds 0.20
TS-07	Fairness	The API's numbers match Fairlearn's at the default threshold within 1e-9, small groups are left out of gaps, and a reject-nothing threshold gives a ratio of 1.0 (FA-11, FA-07, FA-16)
TS-08	API contracts	Every endpoint returns the documented fields; unknown parameters give 400; unknown loans give 404; training loans are never served (AP-04, MD-14)
TS-09	Self-check	The API refuses to start when the model file does not match the stored probabilities (MD-13)
TS-10	Reproducibility	Two runs with the same seed give the same metadata and the same probabilities (MD-04)
TS-11	Smoke	After `make up`, `/health` returns 200, `/optimize` returns a result, and the dashboard page loads

9.5 Security and data handling

DT-08 No authentication in v1 (AP-05), so the services must not be exposed beyond the local machine or a private network. Query parameters are validated and unknown ones rejected. The model is stored in LightGBM's text format, never pickle. The dataset contains no personal information, only anonymized loan identifiers.

DT-09 Data licensing. Raw files, processed tables, and the holdout table all contain Freddie Mac loan-level data and are never committed to the repository or published. Freddie Mac requires a licensing agreement for commercial redistribution of the data. The v1 target is therefore local use. Any public demo is a stretch goal that needs a review of Freddie Mac's current data terms first.

9.6 Non-goals

Authentication, user accounts, automated retraining, a model registry, experiment tracking, scoring of new user-entered loans, and group-specific mitigation (Section 1.1 and Section 5.8).

Appendix A. Traceability to the proposed changes

Proposed change	Where it is specified
A. Interactive what-if business optimizer	Sections 3.3 and 4; `/optimize` in Section 7; Business Optimizer tab in Section 8
B. Fairness and bias engine	Section 5; `/fairness` in Section 7; Fairness and Bias Audit tab in Section 8
C. Drift monitoring with PSI	Section 6; `/drift` in Section 7; Drift Monitor tab in Section 8
D. Dataset swap	Section 2 (Freddie Mac data, label, features, splits)

The requirement ID prefixes are DP (data pipeline), MD (model), OP (optimizer), FA (fairness), DR (drift), AP (API), UI (dashboard), DT (deployment), and TS (tests).

Appendix B. Amendment register

These changes were made to earlier sections as later ones were written. Each is stated in the section where it was made, and is collected here so nothing is missed.

Change	Made in	Affects
`download_data.py` replaced by a manual download and `ingest_data.py` (the diagram in Section 1.2 shows the final name)	2.8	Sections 1.2, 9
Holdout table also holds the model's feature columns	3.7	DP-11
Default false-positive cost set to 2% (illustrative)	4.6	MD-07, OP-04
Number of borrowers capped at 2 as a model feature	5.9	DP-09
Holdout table also holds the four fairness group columns	5.9	DP-11

Appendix C. Items to confirm

These are the assumptions most worth a second look before implementation.

- **Default false-positive cost (2%)** is a placeholder, not an estimate from data.
- **HARP loans are excluded** in v1, which narrows the population.
- **The label window (24 months)** and the credit-event codes in DP-07 are a judgment call, and every label depends on them.
- **Performance data cutoff** for the chosen release must be confirmed at ingestion (DP-03), so that the 2020–21 stress vintages have a full 24-month window.
- **Pandemic forbearance** affects delinquency in the stress vintages (DP-08).
- **Selection bias:** only accepted loans are observed (Section 1.4).
- **Fairness proxies** are not protected classes, and flagged gaps are not proof of discrimination (FA-01).
- **Public hosting** of any loan-level output needs a check of Freddie Mac's data terms (DT-09).